AI speed. Human accountability.

CREDSHIELDS · SMART CONTRACT AUDIT

Smart Contract Audit

BitSafe

This report presents the findings of a smart contract security audit conducted against Funtoken CoinFlip Contracts. Findings were identified through manual review of the contract source by the CredShields' security team.

1	2	3	10	15	31
CRITICAL	HIGH	MEDIUM	LOW	INFO	TOTAL FINDINGS

CredShields Technologies Pte. Ltd. · Singapore
credshields.com · info@credshields.com

CONFIDENTIAL

/ CONTENTS

Table of Contents

01 Executive Summary

02 Vulnerability Summary

03 Detailed Findings

C001 Unauthenticated Cross-Chain Compose Enables Permanent Fee-Poisoning and Drain of Arbitrary User Balances — CRITICAL

H001 Bridge Intent Fields Are Not Bound to Player Authorization (Full Deposit Diversion) — HIGH

H002 Global usedServerSeeds Registry Lets Anyone Burn a Revealed Server Seed to Permanently Brick Settlement (Loss-Escape / Denied Payout) — HIGH

M001 Bridge Fee Priced From an Unsigned, Unbounded, Caller-Supplied Ratio — MEDIUM

M002 Live Deposit Path Enforces No LayerZero Compose-Gas Floor (Stalled Deposits) — MEDIUM

M003 defundBankroll Ignores Outstanding Reservations and Can Extract Committed Funds — MEDIUM

L001 Amount Below the Conversion Rate Is Fully Lost and Emits a Zero-Value Bridge — LOW

L002 Compose Revert Before Finalization Locks Minted FUN in the Ownerless Composer — LOW

L003 Failed-Credit Refund Returns the Full Amount With No Fee Charged (Fee Dodge) — LOW

L004 Caller-Controlled depositForWager Defeats the Fee-Waiver Playthrough Gate — LOW

L005 setPayoutBps Has No Lower Bound; a Zero Payout Is Accepted — LOW

L006 Settlement Trusts Operator-Supplied Outcomes; Dispute Only Pauses, No Recovery — LOW

L007 Globally-Namespaced sessionId Makes Registration Front-Runnable — LOW

L008 Lowering maxFeeOwedLD Does Not Re-Clamp Already-Accrued Fees — LOW

L009 Per-Game Reservation Against a Shared Bankroll Allows Cross-Game Over-Commitment — LOW

L010 No Upper Bound on Session Bet Count (Unbounded Loop) — LOW

I001 Vestigial Signature Scaffolding (`_hashIntent` Computed but Never Verified) — INFO

I002 No Sweep/Receive; Lossless, Non-Rebasing FUN Is an Unstated Requirement — INFO

I003 Fee Waiver Is Unreachable When `wagerRequired` Is Zero — INFO

I004 _hashCommitment Omits the Contract Address (Cross-Instance Replay) — INFO

I005 Self-Submitted Bridge Is Charged the Native Fee and an Owed Fee (Double Charge) — INFO

I006	Per-Bet Maximum Does Not Bound Per-Session Aggregate Stake	INFO
I007	Administrative Parameter Setters Lack Sanity Bounds	INFO
I008	Permissive As-Deployed Defaults Require Post-Deployment Configuration	INFO
I009	Missing NatSpec Documentation Across All Contracts	INFO
I010	On-Chain Commit-Reveal Does Not Enforce Provable Fairness; Seed Integrity Rests on the Trusted Off-Chain Generator	INFO
I011	No Session Cancel/Expiry Strands Stake and Reservation Indefinitely	INFO
I012	Sub-Precision Dust Mishandled on the OFT Deposit and Withdrawal Paths	INFO
I013	Registration Reservation Is Looser Than the Per-Bet Stake Lock (Liveness Window)	INFO
I014	Public Constants Can Be Made Private	INFO
I015	Combined Revert Conditions Can Be Split for Runtime Savings	INFO

– Lead Reviewers & Signatories

/ 01: THE PICTURE AT A GLANCE

Executive Summary

This report summarizes the security audit engagement for BitSafe's Funtoken CoinFlip Contracts. Our assessment identified a total of **31 findings**, categorized into 1 Critical, 2 High, 3 Medium, 10 Low, and 15 Informational severity issues.

The **Critical** finding, "Unauthenticated Cross-Chain Compose Enables Permanent Fee-Poisoning and Drain of Arbitrary User Balances," represents a severe risk due to improper authentication that could lead to significant financial loss. The two **High** severity findings, "Bridge Intent Fields Are Not Bound to Player Authorization (Full Deposit Diversion)" and "Global usedServerSeeds Registry Lets Anyone Burn a Revealed Server Seed to Permanently Brick Settlement (Loss-Escape / Denied Payout)," also pose serious threats, potentially enabling unauthorized fund diversion and denial of service for payouts. Addressing these Critical and High severity issues should be prioritized due to their potential impact.

Additionally, 3 Medium severity findings were identified, concerning issues such as missing input validation, denial of service, and business logic flaws. The audit also revealed 10 Low severity and 15 Informational findings, which provide further opportunities for hardening the system and improving code quality. We are pleased to report that following the audit, **all 31 identified findings have been confirmed as fixed**, with no acknowledged items remaining open.

Audited scope

https://github.com/shubhodeep1/fun-token-multi-chain/tree/30f592dd6adb44c00a677cf899578fdd84847794

Retested scope

https://github.com/shubhodeep1/fun-token-multi-chain/tree/ea07ef323eebf06887de46dd58a855d2020091cc

Assessment overview

This smart contract security audit for BitSafe's Funtoken CoinFlip Contracts involved a comprehensive and multi-faceted review of the provided Solidity codebase. Our methodology combined manual code inspection by experienced auditors with automated static analysis tools to identify potential vulnerabilities. A particular focus was placed on critical security flaws, design and business logic issues, and best practices, especially concerning cross-chain interactions, token handling, and the integrity of game mechanics within the CoinFlip system. The audit aimed to ensure the robustness, reliability, and security of the contracts against a broad range of attack vectors.

The engagement identified a total of 31 findings, covering a spectrum of severities. At the highest level, one **critical** vulnerability, titled **"Unauthenticated Cross-Chain Compose Enables Permanent Fee-Poisoning and Drain of Arbitrary User Balances"** (C001), was discovered. This severe issue highlighted a significant risk of unauthenticated cross-chain interaction leading to a complete compromise of user funds and system integrity. Two **high** severity findings were also identified: **"Bridge Intent Fields Are Not Bound to Player Authorization (Full Deposit Diversion)"** (H001), which could lead to unauthorized deposit redirection, and **"Global usedServerSeeds Registry Lets Anyone Burn a Revealed Server Seed to Permanently Brick Settlement (Loss-Escape / Denied Payout)"** (H002), presenting a critical denial-of-service vector impacting settlement integrity.

Three **medium** severity issues were found, primarily concerning input validation and business logic. These included **"Bridge Fee Priced From an Unsigned, Unbounded, Caller-Supplied Ratio"** (M001), posing a risk for fee manipulation, **"Live Deposit Path Enforces No LayerZero Compose-Gas Floor (Stalled Deposits)"** (M002), which could lead to deposits being stalled, and **"defundBankroll Ignores Outstanding Reservations and Can Extract Committed Funds"** (M003), highlighting a potential for incorrect bankroll management.

A substantial number of **low** severity findings (10 in total) were identified, addressing various areas such as missing input validation (**"Amount Below the Conversion Rate Is Fully Lost and Emits a Zero-Value Bridge"** (L001), **"setPayoutBps Has No Lower Bound; a Zero Payout Is Accepted"** (L005)), business logic flaws (**"Failed-Credit Refund Returns the Full Amount With No Fee Charged (Fee Dodge)"** (L003), **"Settlement Trusts Operator-Supplied Outcomes; Dispute Only Pauses, No Recovery"** (L006)), and denial of service concerns (**"Compose Revert Before Finalization Locks Minted FUN in the Ownerless Composer"** (L002), **"Globally-Namespaced sessionId Makes Registration Front-Runnable"** (L007)). Additionally, 15 **informational** findings were noted, offering recommendations for gas optimizations (**"Public Constants Can Be Made Private"** (I014)), improved documentation (**"Missing NatSpec Documentation Across All Contracts"** (I009)), and general best practices related to cryptographic assurances (**"On-Chain Commit-Reveal Does Not Enforce Provable Fairness"** (I010)) and administrative control parameterization.

We are pleased to report that all 31 identified findings, encompassing critical, high, medium, low, and informational severities, have been successfully addressed and fixed by the BitSafe team during the retest phase. This proactive and thorough remediation significantly enhances the overall security posture and reliability of the Funtoken CoinFlip Contracts.

ENGAGEMENT DETAILS

CLIENT	BitSafe
ASSESSMENT TYPE	Smart Contract Security Audit
AUDIT	FunToken Multichain CoinFlip Game
REPORT DATE	14 Jul 2026
START DATE	11 Jun 2026
END DATE	19 Jun 2026
RETEST DATE	07 Aug 2026
TOTAL FINDINGS	31
REVIEWER	Shashank (shashank@credshields.com) · CredShields

/ FINDING SEVERITY DISTRIBUTION

1	2	3	10	15
CRITICAL	HIGH	MEDIUM	LOW	INFO



/ 02: AT A GLANCE

Vulnerability Summary

Every finding from this assessment, ordered by severity. Follow an ID for the full write-up.

ID	VULNERABILITY	SEVERITY	CATEGORY	STATUS
C001	Unauthenticated Cross-Chain Compose Enables Permanent Fee-Poisoning and Drain of Arbitrary User Balances	CRITICAL	Improper Authentication (SCWE-034)	✓ FIXED
H001	Bridge Intent Fields Are Not Bound to Player Authorization (Full Deposit Diversion)	HIGH	Improper Authentication (SCWE-106)	✓ FIXED
H002	Global usedServerSeeds Registry Lets Anyone Burn a Revealed Server Seed to Permanently Brick Settlement (Loss-Escape / Denied Payout)	HIGH	Denial of Service (SCWE-087)	✓ FIXED
M001	Bridge Fee Priced From an Unsigned, Unbounded, Caller-Supplied Ratio	MEDIUM	Missing Input Validation (SCWE-013)	✓ FIXED
M002	Live Deposit Path Enforces No LayerZero Compose-Gas Floor (Stalled Deposits)	MEDIUM	Denial of Service (SCWE-094)	✓ FIXED
M003	defundBankroll Ignores Outstanding Reservations and Can Extract Committed Funds	MEDIUM	Business Logic (SCWE-125)	✓ FIXED
L001	Amount Below the Conversion Rate Is Fully Lost and Emits a Zero-Value Bridge	LOW	Missing Input Validation (SCWE-091)	✓ FIXED
L002	Compose Revert Before Finalization Locks Minted FUN in the Ownerless Composer	LOW	Denial of Service (SCWE-146)	✓ FIXED
L003	Failed-Credit Refund Returns the Full Amount With No Fee Charged (Fee Dodge)	LOW	Business Logic (SCWE-083)	✓ FIXED
L004	Caller-Controlled depositForWager Defeats the Fee-Waiver Playthrough Gate	LOW	Business Logic (SCWE-013)	✓ FIXED
L005	setPayoutBps Has No Lower Bound; a Zero Payout Is Accepted	LOW	Missing Input Validation (SCWE-083)	✓ FIXED
L006	Settlement Trusts Operator-Supplied Outcomes; Dispute Only Pauses, No Recovery	LOW	Business Logic (SCWE-017)	✓ FIXED
L007	Globally-Namespaced sessionId Makes Registration Front-Runnable	LOW	Denial of Service (SCWE-087)	✓ FIXED
L008	Lowering maxFeeOwedLD Does Not Re-Clamp Already-Accrued Fees	LOW	Business Logic (SCWE-083)	✓ FIXED
L009	Per-Game Reservation Against a Shared Bankroll Allows Cross-Game Over-Commitment	LOW	Business Logic / Denial of Service (SCWE-125)	✓ FIXED
L010	No Upper Bound on Session Bet Count (Unbounded Loop)	LOW	Denial of Service / Gas (SCWE-148)	✓ FIXED

ID	VULNERABILITY	SEVERITY	CATEGORY	STATUS
I001	Vestigial Signature Scaffolding (`_hashIntent` Computed but Never Verified)	INFO	Informational (SCWE-056)	✓ FIXED
I002	No Sweep/Receive; Lossless, Non-Rebasing FUN Is an Unstated Requirement	INFO	Informational (SCWE-110)	✓ FIXED
I003	Fee Waiver Is Unreachable When `wagerRequired` Is Zero	INFO	Informational / Business Logic (SCWE-083)	✓ FIXED
I004	_hashCommitment Omits the Contract Address (Cross-Instance Replay)	INFO	Informational (SCWE-131)	✓ FIXED
I005	Self-Submitted Bridge Is Charged the Native Fee and an Owed Fee (Double Charge)	INFO	Informational (SCWE-083)	✓ FIXED
I006	Per-Bet Maximum Does Not Bound Per-Session Aggregate Stake	INFO	Informational (SCWE-083)	✓ FIXED
I007	Administrative Parameter Setters Lack Sanity Bounds	INFO	Informational (SCWE-083)	✓ FIXED
I008	Permissive As-Deployed Defaults Require Post-Deployment Configuration	INFO	Informational (SCWE-145)	✓ FIXED
I009	Missing NatSpec Documentation Across All Contracts	INFO	Informational (SCWE-015)	✓ FIXED
I010	On-Chain Commit-Reveal Does Not Enforce Provable Fairness; Seed Integrity Rests on the Trusted Off-Chain Generator	INFO	Cryptography (SCWE-024)	✓ FIXED
I011	No Session Cancel/Expiry Strands Stake and Reservation Indefinitely	INFO	Denial of Service (SCWE-140)	✓ FIXED
I012	Sub-Precision Dust Mishandled on the OFT Deposit and Withdrawal Paths	INFO	Business Logic (SCWE-124)	✓ FIXED
I013	Registration Reservation Is Looser Than the Per-Bet Stake Lock (Liveness Window)	INFO	Business Logic (SCWE-083)	✓ FIXED
I014	Public Constants Can Be Made Private	INFO	Gas Optimization	✓ FIXED
I015	Combined Revert Conditions Can Be Split for Runtime Savings	INFO	Gas Optimization	✓ FIXED

/ 03: DETAILED FINDINGS

31 findings across the assessment.

Findings are ordered by severity.

C001 · IMPROPER AUTHENTICATION (SCWE-034)

CRITICAL

FIXED

Unauthenticated Cross-Chain Compose Enables Permanent Fee-Poisoning and Drain of Arbitrary User Balances

DESCRIPTION

The destination compose handler that finalizes every bridged deposit authenticates only the local transport and the local token: it confirms the call came from the messaging endpoint and that the delivered token is the FUN token, but it never checks the source chain identifier or the source-chain sender, both of which are carried in the compose envelope. The router is therefore not a trust boundary, and any message the FUN token mesh delivers, from any peer chain and sent by anyone, is accepted with a fully attacker-chosen payload.

An attacker exploits this by calling the token's public cross-chain send directly, through the adapter on the source chain or the token on any peer chain, addressing the message to the composer and embedding a forged deposit payload that names a victim as the beneficiary and an arbitrarily large owed fee. The bridged principal can be zero or sub-precision, so the attack costs only gas. On delivery, the composer accepts the forged payload because it performs no source authentication; the destination owed-fee ceiling is left unset at deployment, where a zero ceiling is treated as no limit, so the fee is not clamped; and the lobby's bridge-credit routine then adds the attacker-chosen owed fee to the victim's account and inflates the victim's wager requirement, both taken verbatim from the forged payload.

The victim can never recover the funds. Their owed fee is collected in full from any withdrawal and forwarded to the fee recipient, so every withdrawal returns nothing, and the fee-waiver path cannot clear it: it never fires when the wager requirement is zero, and the same payload inflates the wager requirement to an unreachable value. The net effect is that any user's entire balance, including funds deposited well before the attack, is permanently frozen and drained to the fee recipient. The same fee-poisoning is also reachable through the router itself, because the owed fee is derived from a caller-supplied price ratio that the router validates only for being non-zero.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunLobbyComposer.sol#L86-L113
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunGameLobby.sol#L155-L204
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunGameLobby.sol#L515-L527

BUSINESS IMPACT

Any user's entire lobby balance, including funds deposited before the attack, can be permanently frozen and routed to the fee recipient on withdrawal, with no recovery path for the victim. Exploitation is permissionless, requires no meaningful capital, affects any account, and the as-deployed configuration with no fee ceiling is the maximally vulnerable one. Because the drained funds flow to the fee recipient rather than the attacker, this is large-scale forced loss and griefing of user funds rather than direct attacker profit. The same unauthenticated-compose gap additionally enables the locked-funds and fee-free-refund issues documented separately, and a fee ceiling configured later only bounds the per-victim loss without removing the attack.

RECOMMENDED NEXT STEP

Authenticate the cross-chain source inside the compose handler: require the source chain identifier to equal the trusted source endpoint and the source-chain sender to equal the trusted router, both configured at deployment or by the owner, so the router becomes a real trust boundary. Independently, bind the credited beneficiary to the caller's own authorization or a player signature so that no third party can name an arbitrary recipient, bound the owed fee against the bridged amount, change the destination fee ceiling so that an unset ceiling rejects rather than disables the cap and configure it at deployment, and reconsider permitting an

owed fee to persist against an account that carries no wager obligation so an injected fee cannot become permanently unclearable.

RETEST

Fixed. The lzCompose entry now performs source authentication at _validateTrustedSource before any crediting.

H001 · IMPROPER AUTHENTICATION (SCWE-106)

HIGH

Bridge Intent Fields Are Not Bound to Player Authorization (Full Deposit Diversion)

✓ FIXED

DESCRIPTION

The router pulls FUN from the transaction sender but takes the destination beneficiary from the intent's recipient field, which is never bound to the sender and never covered by any signature the router verifies. A domain-separated intent hash is computed but is used only for the nonce event; there is no signature recovery anywhere. In the intended sponsored flow the player authorizes only an account delegation, not the specific intent, so the router has no proof the player consented to this particular bridge. Whenever an actor can cause the transaction sender to be the victim the relayer the victim trusted, anyone who reuses a still-valid delegation authorization, or anyone if a permissionless delegate is left installed on the victim's account that actor can rewrite the recipient field and divert the victim's entire bridged amount to an account they control. The enabling layers are out of scope, but the router's lack of a per-intent signature is what turns the blast radius from nil into full theft.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L194-L211
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L333-L351

BUSINESS IMPACT

Full loss of the bridged amount for an affected transaction, redirected to an attacker-chosen account. Exploitable by a malicious or compromised relayer, or by anyone who can execute as the victim's account.

RECOMMENDED NEXT STEP

Require a player signature over the full intent and verify it on-chain, so that even an actor executing as the victim cannot forge consent to a malicious intent. Submission can remain permissionless once the signature is the authorization; a submitter whitelist is not a substitute and introduces censorship and liveness risk. If deposits on behalf of others are not a product feature, alternatively require the recipient to equal the caller.

RETEST

Fixed. This issue is fixed by adding require check of _intent.recipient \== msg.sender.

H002 · DENIAL OF SERVICE (SCWE-087)

HIGH

FIXED

Global usedServerSeeds Registry Lets Anyone Burn a Revealed Server Seed to Permanently Brick Settlement (Loss-Escape / Denied Payout)

DESCRIPTION

Every revealed server seed is recorded in a single global, permissionless, never-cleared registry (usedServerSeeds), and registerSession accepts a fully player-authored commitment with no operator signature. Because the registry is keyed on the raw seed while a session's commitment immutably binds one specific seed S, any actor who learns S from the operator's own pending on-chain reveal, whose settleSession or prepareSessionCashout calldata carries S can register a throwaway session that legitimately commits to the same S and call the permissionless prepareSessionCashout to set usedServerSeeds[S] \= true before the operator's transaction is mined. The operator's real settleSession then reverts ServerSeedReused and can never succeed, because the victim session's commitment can only ever be revealed with S; prepareSessionCashout on the real session reverts identically. The victim session is left with revealedServerSeed \== 0, so after sessionExpirySeconds the player cancels it and reclaims the full stake. A losing player therefore escapes the loss (settle the wins, brick-and-cancel the losses) for a coinflip with no downside, and the same primitive lets an attacker brick a winning victim's session to deny their payout. The throwaway session costs one minimum bet, which is zero because minBetLD has no lower bound. This defect was introduced by the retest remediations the usedServerSeeds seed-reuse guard and the cancelSession/prepareSessionCashout cashout flow.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/2774391dc93aa2e0ce0d53040d3f0dba619d9be0/contracts/FunCoinflip.sol#L281

BUSINESS IMPACT

A losing player can permanently brick their own settlement and reclaim their stake via cancelSession, turning the game strictly positive expected value against the house (bet with no downside) and draining the bankroll over time; the same primitive lets an attacker permanently brick a winning victim's session and deny their payout. There is no on-chain recovery usedServerSeeds is never cleared and no admin path can un-mark a poisoned seed, so a bricked session is permanently unsettleable. Precondition: the attacker must observe S before the operator's reveal is mined (front-running the reveal), so practical exploitability depends on the destination chain's mempool visibility; the contract-level defect is unconditional.

RECOMMENDED NEXT STEP

Keep the seed-reuse guard, but require operator authorization to consume a seed so no unprivileged actor can set usedServerSeeds[seed] \= true. Require an operator signature on prepareSessionCashout an operator-signed authorization bound to address(this), chainid, player, sessionId, and serverSeed, verified the same way settleSession verifies its batch signature. Do not implement this as a msg.sender \== operatorSigner check: that would force the cold signing key to become a gas-paying transaction sender and break the operatorSigner / settlement-submitter separation. settleSession's unprepared branch already requires the operator batch signature, so with both writers of usedServerSeeds gated, the burn vector is closed while the reuse protection is preserved and submission stays permissionless.

RETEST

Fixed. The seed□poison / loss□escape vulnerability is remediated and verified registerSession now requires an operator ECDSA co□signature (_assertValidRegistrationSignature), so no unprivileged actor can register a session binding a victim's revealed seed, and both writers of usedServerSeeds are gated behind operator authorization. Optional hardening (non□exploitable, not required to close): _hashRegistrationAuthorization carries no domain tag and is structurally identical to _hashCommitment recommend prefixing it with a bytes32 domain□tag constant as defense□in□depth (do not retag the other hashes).

M001 · MISSING INPUT VALIDATION (SCWE-013)

Bridge Fee Priced From an Unsigned, Unbounded, Caller-Supplied Ratio

MEDIUM

✓ FIXED

DESCRIPTION

The owed fee is computed as the total native cost multiplied by a caller-supplied numerator and divided by a caller-supplied denominator, with the only validation being that neither is zero. There is no oracle, no upper or lower bound, and no sanity check against a real native-to-FUN rate. A caller can drive the owed fee down to roughly one unit while the protocol pays the real native cost, leaking revenue and, in the sponsored model, draining whoever funded the native fee; or drive it extremely high to feed the critical account fee-poisoning issue. The destination fee ceiling is only an upper clamp that is disabled when zero, so it neither establishes a floor nor prevents under-reporting. The player commits to no maximum fee, and the reserve component of the cost is also recomputed live, so the value can change between quote and execution.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L273-L285
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L240-L243

BUSINESS IMPACT

The fee a user owes is fully self-determined with no on-chain integrity, enabling fee evasion and protocol-revenue or sponsor-float leakage, and feeding the fee-poisoning path in the critical issue.

RECOMMENDED NEXT STEP

Derive the price on-chain from a trusted source such as an oracle or an owner-set rate, or bind the pricing fields to a player or pricing-authority signature that is verified before use. Add an explicit player-committed maximum fee and bound the computed fee against both that maximum and the bridged amount.

RETEST

Fixed. The caller no longer supplies pricing fields at all the BridgeIntent struct carries only recipient, amountLD, minAmountLD, nonce, expiry, maxFeeLD

M002 · DENIAL OF SERVICE (SCWE-094)

Live Deposit Path Enforces No LayerZero Compose-Gas Floor (Stalled Deposits)

MEDIUM

✓ FIXED

DESCRIPTION

The router takes the caller-supplied LayerZero execution options from the bridge intent and forwards them verbatim into the cross-chain send, funding the resulting native fee from the transaction value, without inspecting, capping, or enforcing a floor on them. Intent validation never examines the options field. This single unvalidated field produces two distinct problems. First, a deposit carries a compose message, so the message type is send-and-call and requires a compose-gas allotment, yet the router enforces no minimum, and the enforced-options configuration for the live destination pathway only constrains the plain send type. A deposit sent with insufficient or absent compose gas mints FUN to the composer but never auto-executes the credit, so the player's lobby balance is never created; the funds are recoverable only because anyone can later trigger the compose manually. A malicious relayer can deliberately under-gas to stall a victim, and an honest tooling bug silently produces stuck deposits, unlike a sibling pathway that was pre-provisioned with a gas floor. Second, the same options field can encode a native-drop instruction that airdrops the destination chain's native gas to an arbitrary address, with that cost priced into the native fee. In the sponsored flow the sponsor supplies the transaction value, so the sponsor pays for the airdrop; combined with the unbounded, caller-supplied fee ratio documented separately, the attacker can drive the owed reimbursement to nearly zero and pocket the drop. This leak is bounded per message by the executor's drop cap and only arises when a sponsor funds player-supplied options without inspecting them, so it is a low-likelihood, hardening-grade consequence of the same missing validation.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L257-L271
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L213-L249
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L189-L211

BUSINESS IMPACT

Deposits on the live pathway can silently fail to credit and require manual recovery, a liveness and griefing exposure that removes the credit-funding guarantee the protocol gave to another pathway. The same unvalidated field additionally lets a sponsor be drained of native value through attacker-encoded native drops, bounded by the executor's drop cap and contingent on the sponsor funding uninspected options and on the separate unbounded-fee-ratio issue. No protocol-held funds or existing user balances are directly lost; the exposure is operator and sponsor liveness and float.

RECOMMENDED NEXT STEP

Build the LayerZero execution options on-chain from owner-set gas parameters instead of accepting them from the caller. This guarantees a compose-gas option with a sane floor, removing the stalled-deposit path, and hard-sets the compose value to zero with no native-drop option, removing the sponsor leak. If caller-supplied options must be retained, enforce a minimum compose gas, reject or cap any native-drop instruction, and bind the owed fee to the bridged amount so an inflated native cost cannot be decoupled from its reimbursement.

RETEST

Fixed. Options are now constructed entirely on-chain: OptionsBuilder.newOptions().addExecutorLzComposeOption(0, uint128(composeGasLimit), 0). The caller cannot supply options (no such field exists). composeGasLimit is owner-set and validated to [MIN_COMPOSE_GAS_LIMIT=50_000, MAX_COMPOSE_GAS_LIMIT=2_000_000], default 200_000.

M003 · BUSINESS LOGIC (SCWE-125)

MEDIUM

defundBankroll Ignores Outstanding Reservations and Can Extract Committed Funds

✓ FIXED

DESCRIPTION

Every registration guarantees that the bankroll covers the maximum payout of all open sessions, and this invariant is preserved across settlements by construction. The owner defund routine is the single operation that escapes that guarantee: it reduces the bankroll while checking only that the requested amount does not exceed the current bankroll, with no awareness of the coinflip's outstanding reservation, which lives in a separate contract the lobby cannot see. The owner can therefore reduce the bankroll below the committed reserve, after which a winning settlement reverts on the bankroll check and the player's payout cannot be paid until the bankroll is restored. Because player stakes are commingled into the bankroll at lock time, the defund routine can also transfer in-flight player stakes out to the owner. The honest case is a recoverable footgun, while a malicious or compromised owner can extract committed reserves and in-flight stakes; the distinguishing risk is that pausing or de-authorizing only freezes settlement whereas this routine extracts funds.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunGameLobby.sol#L318-L330
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunGameLobby.sol#L256-L259

BUSINESS IMPACT

Winning settlements can be stranded, and in the malicious case the owner can withdraw funds that back not-yet-settled players, including their in-flight stakes. No insolvency of existing balances occurs, but pending wins become unpayable until the bankroll is restored.

RECOMMENDED NEXT STEP

Track total outstanding reservations in the lobby through a shared counter that authorized games increment on reservation and decrement on settlement, and require that a defund cannot reduce the bankroll below that total. As a minimum, document and operationally enforce that the bankroll is never reduced below the games' combined reservation, ideally behind a timelock or multisig owner.

RETEST

Fixed. defundBankroll now caps the withdrawal at the unreserved portion: availableDefundAmountLD \= houseBankroll - totalReservedLD, revert if _amountLD > availableDefundAmountLD

L001 · MISSING INPUT VALIDATION (SCWE-091)

LOW

Amount Below the Conversion Rate Is Fully Lost and Emits a Zero-Value Bridge

✓ FIXED

DESCRIPTION

The intent validation checks only that the amount is non-zero, not that it survives dust removal. When the amount is smaller than the conversion rate, the dust-removed amount is zero: the router still pulls the full amount and keeps it, the adapter debits nothing, and a zero-value bridge is delivered with a valid compose message. The destination credits a zero balance but still records the owed fee and wager requirement, so the player loses the amount, pays gas, and owes a real fee for a zero-value deposit. In the sponsored flow this can be used to grief a player into fee debt, and it is also the mechanism that lets the critical fee-poisoning attack run with zero principal.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L220-L223

BUSINESS IMPACT

A sub-rate amount is fully lost, and a zero-value deposit still imposes an owed fee and wager state on the recipient.

RECOMMENDED NEXT STEP

Require that the dust-removed amount is at least a configured minimum so that a zero-value bridge cannot be initiated.

RETEST

Fixed. _validateIntent now normalizes first (bridgeAmountLD \= _normalizeBridgeAmount(amount), and rejects bridgeAmountLD < minBridgeAmountLD

L002 · DENIAL OF SERVICE (SCWE-146)

LOW

Compose Revert Before Finalization Locks Minted FUN in the Ownerless Composer

✓ FIXED

DESCRIPTION

FUN is minted to the composer before the compose finalizes. If the compose reverts before the processed-guid flag is set for example on a malformed payload that fails to decode, or on a zero beneficiary the message can never finalize, the recoverable-balance record is never written because it is only set in the catch branch that runs later, and the composer has no owner, pause, or sweep. The minted FUN is therefore permanently locked. This is reachable only through the direct-send bypass today, since the router always emits a well-formed payload, but the stuck funds have no recovery.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunLobbyComposer.sol#L101-L115

BUSINESS IMPACT

A bridged amount can be permanently locked in the ownerless composer with no recovery path, for messages that revert before finalization.

RECOMMENDED NEXT STEP

Removing the source-authentication gap eliminates the reachability. Independently, validate the payload so it never strands funds, and consider a permissionless rescue for messages that cannot be credited. At minimum, document that the composer has no rescue path for stray value.

RETEST

Fixed. Decode/zero-player and unset-cap reverts all occur before processedGuids[_guid] \= true, so such messages are not finalized and remain retriable via LayerZero compose-retry.

L003 · BUSINESS LOGIC (SCWE-083)

LOW

Failed-Credit Refund Returns the Full Amount With No Fee Charged (Fee Dodge)

✓ FIXED

DESCRIPTION

When the lobby credit reverts, the composer parks the full bridged amount as a recoverable balance and the claim routine later pays it out with no fee charged. Any reproducible credit revert therefore becomes a full refund with the owed fee bypassed. Reproducible reverts include an overflow of the wager computation via an inflated wager value supplied through the direct-send bypass, and depositing while the lobby is paused, which converts a transient condition into a fee-free refund instead of a pending credit.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunLobbyComposer.sol#L120-L129

BUSINESS IMPACT

The owed bridge fee can be dodged by forcing a credit revert, and a paused-lobby deposit silently becomes a fee-free refund rather than a retriable credit.

RECOMMENDED NEXT STEP

Close the source-authentication gap to remove the inflated-wager vector, and decide explicitly whether a paused-lobby deposit should refund or remain retriable rather than defaulting to a fee-free refund.

RETEST

Fixed. The inflated wager revert vector is closed by the C001 source authentication fix… residual resolved per recommendation option (b): #348 documents the paused window fee free refund as by design.

L004 · BUSINESS LOGIC (SCWE-013)

LOW

Caller-Controlled depositForWager Defeats the Fee-Waiver Playthrough Gate

✓ FIXED

DESCRIPTION

The wager requirement that gates the fee waiver is built from a caller-supplied field bounded only by not exceeding the bridged amount, with no lower bound and no enforced relationship to the actual deposit or the fee. Whoever builds the intent can set it minimal while depositing a large amount, making the wager requirement negligible so the fee is waived after a single minimum bet. This nullifies the economic purpose of the playthrough multiple, which is to require proportional betting volume in exchange for the gas subsidy. In the honest sponsored flow the relayer sets the field, but a self-funding player or the direct-send bypass controls it. Additionally, because the wager requirement and the cumulative wagered total are lifetime figures that are never reset, once a player's cumulative wagered exceeds their wager requirement, every future deposit sent with a zero wager-deposit value leaves the requirement unchanged and has its fee waived immediately, with no new wagering at all, letting a repeat user bridge fee-free indefinitely.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L230-L233
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunGameLobby.sol#L191

BUSINESS IMPACT

The fee-waiver playthrough requirement can be reduced to near zero, letting a user waive the bridge fee without meaningful betting volume and defeating the gas-subsidy gate.

RECOMMENDED NEXT STEP

Derive the wager requirement from the actually credited amount rather than a separate caller-supplied field, or require the field to equal the bridged amount. If partial-wager deposits are a real feature, require the player to sign the field and bound it.

RETEST

Fixed. The caller□controlled wager field is removed… residual accepted as by□design: #348 documents the lifetime□total waiver.

L005 · MISSING INPUT VALIDATION (SCWE-083)

setPayoutBps Has No Lower Bound; a Zero Payout Is Accepted

LOW

✓ FIXED

DESCRIPTION

The payout configuration setter rejects only values at or above the maximum, so a value of zero is accepted. With a zero payout configured, every subsequent session's winning bets pay nothing and the stakes are silently retained by the house. Owner trust mitigates this, but a configuration mistake degrades the game to a guaranteed loss for players.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L357-L360

BUSINESS IMPACT

A misconfigured zero payout causes winning bets to pay nothing, with stakes retained, until the value is corrected.

RECOMMENDED NEXT STEP

Add a minimum payout floor so that a sub-fair payout, including zero, is rejected.

RETEST

Fixed. setPayoutBps now requires MIN_PAYOUT_BPS (10_000) ≤ _payoutBps < MAX_PAYOUT_BPS (20_000).

L006 · BUSINESS LOGIC (SCWE-017)

LOW

Settlement Trusts Operator-Supplied Outcomes; Dispute Only Pauses, No Recovery

✓ FIXED

DESCRIPTION

Each coinflip session is settled from an operator-signed batch in which the per-bet outcome is supplied by the operator. The settlement routine pays winnings whenever the supplied outcome equals the player's pick and never recomputes the outcome from the revealed seed, even though every input needed to derive it deterministically is already on-chain at settlement time. The on-chain verification exists only in the dispute routine, which recomputes the true outcome from the revealed seed but, on a mismatch, does nothing more than pause the contract. There is no clawback, no refund, and no reversal; the session remains marked settled and the owner can simply unpause with the fraudulent settlement standing. Consequently, an operator key that is compromised or dishonest can sign losing outcomes for open sessions and convert staked funds into house funds, or sign winning outcomes to drain the bankroll, with the only on-chain consequence being a reversible freeze.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L255-L258
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L335-L346

BUSINESS IMPACT

A single operator-key compromise or a dishonest house can confiscate player stakes or drain the bankroll across all open sessions, with no on-chain recovery. The dispute mechanism reduces to an event and a pause that the owner can lift.

RECOMMENDED NEXT STEP

Derive each outcome on-chain inside the settlement routine from the revealed seed, the stored client seed, and the bet nonce, and reject any batch whose declared outcomes do not match. This makes the commit-reveal binding and reduces the operator signature to authorizing only when the seed is revealed, not what the outcomes are. As defense in depth, make the dispute remediating rather than pause-only, add a challenge window before settlement finalizes, and consider a threshold or multisig signer.

RETEST

Fixed
Settlement now re-derives every outcome on-chain actualOutcome \= _computeOutcome(serverSeed, clientSeed, nonce) (L393) and reverts OutcomeMismatch if bet.outcome != actualOutcome

L007 · DENIAL OF SERVICE (SCWE-087)

Globally-Namespaced sessionId Makes Registration Front-Runnable

LOW

✓ FIXED

DESCRIPTION

Sessions are keyed by identifier alone rather than by player, so a mempool watcher can observe a victim's pending registration and register the same identifier first, causing the victim's transaction to revert as already registered. The front-runner gains nothing from doing so. The commitment binds both the session identifier and the player, so a copied commitment cannot be settled under a different registrant, and any session the attacker registers is bound to the attacker with its outcomes determined by a server seed the attacker cannot predict, which is an ordinary even-odds game played with the attacker's own funds. The attack therefore cannot redirect a victim's payout or take over the victim's session; its only effect is to deny the victim a specific identifier and force a retry with a fresh one. The cost falls entirely on the attacker: when a non-zero minimum bet is configured the griefer must lock real stake in a session it controls, and if that session was registered with a commitment the operator will never reveal, the stake is stranded permanently because there is no cancel path. In the as-deployed configuration, however, the minimum bet defaults to zero, so the griefer can register zero-stake bets that lock nothing, making the denial of service free and repeatable against a targeted victim. The victim never loses funds, because their registration reverts atomically.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L159-L162
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L476-L484

BUSINESS IMPACT

A front-runner can deny a targeted victim a specific session identifier and force repeated retries, and can do so for free in the as-deployed configuration where the minimum bet is zero. The attacker gains nothing and cannot hijack or settle the victim's session, so there is no fund loss to either party beyond the griefer's own stranded stake when a non-zero minimum bet is configured. The net effect is a liveness nuisance with no attacker profit motive.

RECOMMENDED NEXT STEP

Key sessions by the registering account together with the identifier, or otherwise bind the identifier to the player at registration, so that two players cannot collide on the same identifier and a front-runner cannot pre-empt another account's chosen identifier. Configuring a non-zero minimum bet additionally removes the free-griefing case by restoring a real cost to each spurious registration.

RETEST

Fixed. Sessions are now keyed mapping(address player \=> mapping(uint256 sessionId \=> Session))

L008 · BUSINESS LOGIC (SCWE-083)

Lowering maxFeeOwedLD Does Not Re-Clamp Already-Accrued Fees

LOW

✓ FIXED

DESCRIPTION

The fee-ceiling setter writes the new ceiling without iterating accounts, and the ceiling only bounds accrual at credit time, not the stored value. The withdrawal routine collects the live stored owed fee, not the current ceiling, so an account whose fee accrued under a higher ceiling is still charged above a subsequently lowered ceiling. This is an owner-observable surprise rather than a player-exploitable issue.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunGameLobby.sol#L370-L381
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunGameLobby.sol#L490-L495

BUSINESS IMPACT

After lowering the ceiling, accounts can still be charged fees above it from amounts accrued earlier, contrary to the apparent meaning of the ceiling.

RECOMMENDED NEXT STEP

Clamp the collected fee to the current ceiling at withdrawal time, or document that the ceiling applies only at accrual time.

RETEST

Fixed. _prepareWithdrawal now clamps the stored fee to the current ceiling at withdrawal time: if (feeClampLD != 0 && feeOwedLD > feeClampLD) { feeOwedLD \= feeClampLD; account.feeOwed \= feeClampLD; } (L695-699) before computing feeCollectedLD

L009 · BUSINESS LOGIC / DENIAL OF SERVICE (SCWE-125)

LOW

Per-Game Reservation Against a Shared Bankroll Allows Cross-Game Over-Commitment

✓ FIXED

DESCRIPTION

The reservation counter is per-game state, but the registration solvency check compares it against the whole shared bankroll without any term for other games' reservations. With two or more bankroll-backed games sharing one lobby, each game independently passes its own check against the same pool, so the sum of reservations can exceed the bankroll. The over-commitment is silent at registration and surfaces only later, when winning settlements across games compete for the pool and the second one reverts on the bankroll check. This reintroduces the cross-game settlement-ordering failure that does not occur within a single game. It is not reachable today because only one bankroll-backed game exists, but the authorization mechanism already supports multiple games, so it activates the moment a second one is added.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L171-L177
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L100

BUSINESS IMPACT

With multiple bankroll-backed games, combined reservations can exceed the shared bankroll, stranding some winning settlements depending on order. Latent until a second game is authorized.

RECOMMENDED NEXT STEP

Track total reservations across all games in the lobby through a shared counter and check registrations against the unreserved bankroll, or allocate a dedicated bankroll per game. Do not deploy a second bankroll-backed game until aggregate reservation accounting exists.

RETEST

Fixed. The reservation is a single shared lobby counter. Registration checks lobby.totalReservedLD() + maxPayoutTotalLD ≤ houseBankroll + totalStakedLD

L010 · DENIAL OF SERVICE / GAS (SCWE-148)

No Upper Bound on Session Bet Count (Unbounded Loop)

LOW

✓ FIXED

DESCRIPTION

Session registration accepts a bet array whose length is only checked to be non-zero, with no upper bound. Both the registration preview and the stake-locking routine iterate the entire array, and the stake-locking routine makes an external lobby call for every bet. A caller supplying a very large array pays the gas and risks exceeding the block gas limit, which makes their own registration fail; the same bet count must later be processed at settlement, so an over-large session that did register can also be harder to settle. The harm is borne by the caller with no third-party impact, but the missing bound is a latent denial-of-service and compounds the stranded-reservation issue for abandoned sessions.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L164-L168
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L427-L435

BUSINESS IMPACT

A session with an excessive bet count can exceed the block gas limit at registration or settlement, causing the caller's own transactions to fail and, for a registered session, complicating settlement. Self-inflicted, with no third-party harm.

RECOMMENDED NEXT STEP

Introduce a maximum bet count per session and reject registrations above it.

RETEST

Fixed. Registration reverts BetCountExceedsMaximum when betCount > MAX_BETS_PER_SESSION (256)

I001 · INFORMATIONAL (SCWE-056)

Vestigial Signature Scaffolding (`_hashIntent` Computed but Never Verified)

INFO

✓ FIXED

DESCRIPTION

The intent struct carries a nonce and expiry, and the contract computes a domain-separated intent hash binding the contract address, chain identifier, sender, and every field, which is the exact shape of a signed meta-transaction. Yet nothing recovers or verifies a signature; the hash is used only for the nonce event. This is the structural smell behind the authorization and pricing issues. If account-delegation is the only intended authorization, it should be documented; if a player signature was intended, its absence is the real defect.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L333-L351
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L195-L196

BUSINESS IMPACT

Indicates an intended-but-missing signature check; confirming intent resolves whether the authorization and pricing findings are by design or defects.

RECOMMENDED NEXT STEP

wire signature recovery over the existing intent hash to make the scaffolding load-bearing,

RETEST

Fixed. Per□intent consent is now enforced on□chain by the new in□scope FunBatchCallDelegation, whose execute() gates on msg.sender == address(this) only the account's own signed transaction can run a batch, so a relayer or third party can no longer force a bridge on the user's behalf. Note: consent is achieved via self□execution only, which removes relayer□sponsored / gasless bridging confirm this tradeoff is intentional and update the stale FunBridgeSponsorRouter NatSpec accordingly.

I002 · INFORMATIONAL (SCWE-110)

INFO

FIXED

No Sweep/Receive; Lossless, Non-Rebasing FUN Is an Unstated Requirement

DESCRIPTION

The bridging flow assumes the FUN token is a standard, non-fee-on-transfer, non-rebasing token, consistent with the underlying adapter's documented assumption of lossless transfers. A fee-on-transfer or rebasing token would break accounting and could cause the adapter to pull more than the router holds, leading to denial of service. There is also no sweep or receive path, so any native value force-sent to the contract is locked, which is negligible but worth noting.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L203-L209

BUSINESS IMPACT

A non-standard FUN token would break accounting or availability. The requirement is currently implicit.

RECOMMENDED NEXT STEP

Document explicitly that FUN must be a standard, lossless, non-rebasing token, and consider an owner sweep for stray balances.

RETEST

Fixed. Router now has receive() (L145) and sweep() for ERC20+native (L277-298); composer has rescue()

I003 · INFORMATIONAL / BUSINESS LOGIC (SCWE-083)

Fee Waiver Is Unreachable When `wagerRequired` Is Zero

INFO

✓ FIXED

DESCRIPTION

An owed fee accrued while the wager requirement is zero can never be waived, because the waiver routine early-returns when the requirement is zero. This occurs for direct deposits, which carry no wager requirement, and whenever the wager multiple is zero. Whether this is a defect depends on intent; owing the fee when no wager is committed is a defensible default, so this is flagged to confirm intent rather than asserted as a bug. It is also the trap that makes the critical account fee-poisoning issue durable.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunGameLobby.sol#L515-L527

BUSINESS IMPACT

A fee accrued with no wager requirement is permanently unclearable except by forfeiting balance on withdrawal, which is what makes the fee-poisoning issue durable.

RECOMMENDED NEXT STEP

Confirm the intended semantics; if a fee with no wager obligation should be clearable, adjust the waiver condition so such a fee does not become permanently stuck.

RETEST

Fixed. _clearFeeIfWaived still early-returns on wagerRequired \== 0 (FunGameLobby.sol), but the dangerous state is now unreachable: a fee is added only when amount > 0, and the same credit adds wagerMultiple * amount ≥ amount to the requirement. You can no longer hold feeOwed > 0 with wagerRequired \== 0 via the bridge path.

I004 · INFORMATIONAL (SCWE-131)

_hashCommitment Omits the Contract Address (Cross-Instance Replay)

INFO

✓ FIXED

DESCRIPTION

The commitment hash binds the chain identifier, session identifier, player, and seeds, but not the contract address, whereas the settlement batch hash does include the contract address. Two co-deployed game instances on one chain therefore share the commitment domain, so a commitment and reveal pair is replayable across them. The practical impact is negligible today given a single instance, and because settlement still requires an instance-specific operator signature.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L476-L484

BUSINESS IMPACT

Commitment domain is not separated per instance, allowing cross-instance replay in a hypothetical multi-instance deployment. Latent.

RECOMMENDED NEXT STEP

Include the contract address in the commitment hash for domain-separation parity with the settlement batch hash.

RETEST

Fixed. _hashCommitment now includes address(this): keccak256(abi.encode(address(this), block.chainid, sessionId, player, clientSeed, serverSeed))

I005 · INFORMATIONAL (SCWE-083)

Self-Submitted Bridge Is Charged the Native Fee and an Owed Fee (Double Charge)

INFO

✓ FIXED

DESCRIPTION

In the self-submitted path where the sender is the player, the player funds the native fee directly through the transaction value and also accrues an owed fee that is meant to reimburse a separate sponsor for fronting that native cost. Because there is no separate sponsor in this path, the player is effectively charged twice for the same gas. The player arguably consents by building the intent, but the sponsor-versus-self fee routing is undocumented.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L189-L195

BUSINESS IMPACT

A self-submitting player pays both the native fee and an owed fee for the same bridge, contrary to the reimbursement intent of the owed fee.

RECOMMENDED NEXT STEP

Document the two paths clearly, or waive the owed fee when the payer is the recipient, since no sponsorship occurred.

RETEST

Fixed. New bridgeSelf() entrypoint calls _bridge(..., true); _computeFeeOwedLD returns 0 when _applySelfSubmitWaiver.

I006 · INFORMATIONAL (SCWE-083)

Per-Bet Maximum Does Not Bound Per-Session Aggregate Stake

INFO

✓ FIXED

DESCRIPTION

Bet-amount validation enforces the maximum-bet ceiling on each individual bet, but the per-session total stake is accumulated with no aggregate ceiling, so a single session can stake many maximum-sized bets whose sum far exceeds the maximum bet. Total exposure is still bounded by the bankroll reservation, so there is no solvency impact, but if the maximum-bet parameter is intended as a per-player or per-session exposure limit, it is not enforced at the session level. Whether this is a defect depends on the intended meaning of the maximum-bet parameter.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L410-L425

BUSINESS IMPACT

A player can take on per-session and per-player exposure far above the per-bet maximum by combining many bets, so the maximum-bet parameter does not cap session-level exposure. No solvency impact.

RECOMMENDED NEXT STEP

If a session-level or player-level exposure cap is intended, enforce a maximum on the aggregate staked amount in addition to the per-bet check; otherwise document that the maximum-bet parameter is per-bet only.

RETEST

Fixed. New maxSessionStakeLD + setMaxSessionStakeLD (FunCoinflip.sol); registration reverts SessionStakeAboveMaximum when totalStakedLD > maxSessionStakeLD

I007 · INFORMATIONAL (SCWE-083)

Administrative Parameter Setters Lack Sanity Bounds

INFO

✓ FIXED

DESCRIPTION

Several owner-only configuration setters accept any value with no minimum or maximum bound. The wager-multiple setter accepts zero, which makes subsequent bridge credits accrue no playthrough requirement, and accepts arbitrarily large values, which would impose an unreachable wager bar on subsequent depositors. The outbound-reserve setter accepts any value with no ceiling, inflating the owed-fee computation, and accepts zero, which removes the reserve surcharge entirely; the destination fee ceiling only clamps the result downward. These are owner-controlled footguns rather than externally exploitable issues, but the absence of bounds means a single mistaken configuration silently degrades fee or wager behaviour.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunGameLobby.sol#L383-L394
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L145-L156

BUSINESS IMPACT

A misconfigured wager multiple or outbound reserve silently alters fee or wager behaviour for all subsequent users, with no on-chain guardrail.

RECOMMENDED NEXT STEP

Add reasonable minimum and maximum bounds to these setters, consistent with the bound already present on the payout setter.

RETEST

Fixed. setWagerMultiple bounded [1, 1000] via _validateWagerMultiple (FunGameLobby.sol) zero and runaway values rejected.

I008 · INFORMATIONAL (SCWE-145)

Permissive As-Deployed Defaults Require Post-Deployment Configuration

INFO

✓ FIXED

DESCRIPTION

The lobby is deployed with only its token and owner, leaving the fee ceiling, the minimum bet, and the maximum bet all at zero and the bankroll empty. A zero fee ceiling is treated as uncapped, a zero minimum bet allows zero-value bets, and a zero maximum bet disables the per-bet ceiling. The system is therefore in its most permissive state immediately after deployment and depends on the owner configuring these parameters and funding the bankroll before use. The uncapped fee ceiling in particular is the precondition for the critical account fee-poisoning issue.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunGameLobby.sol#L114-L124

BUSINESS IMPACT

Immediately after deployment the fee ceiling is uncapped and bet bounds are disabled, so the protocol is unsafe to use until the owner explicitly configures these parameters. Misordered or forgotten configuration leaves the maximally permissive state live.

RECOMMENDED NEXT STEP

Set the fee ceiling, minimum bet, and maximum bet in the constructor or a guarded initializer, and document the required post-deployment configuration and its ordering.

RETEST

Fixed. The critical precondition (uncapped fee ceiling) is eliminated: the lobby constructor reverts on maxFeeOwedLD \== 0 (FunGameLobby.sol), and the composer rejects an unset cap (FunLobbyComposer.sol).

I009 · INFORMATIONAL (SCWE-015)

Missing NatSpec Documentation Across All Contracts

INFO

✓ FIXED

DESCRIPTION

None of the in-scope contracts carry NatSpec documentation. There are no contract-level, function-level, parameter, or return doc comments anywhere in the six files, which is why the automated scan reports the full family of missing-NatSpec items (missing author, dev, notice, inheritdoc, scope-block comments, and public-variable descriptions). The absence of NatSpec does not affect runtime behaviour, but it makes the code harder for integrators, reviewers, and future maintainers to understand and raises the chance of misuse of the privileged and cross-chain entry points.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L46
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunGameLobby.sol#L33
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L37
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunLobbyComposer.sol#L22
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunOFT.sol#L37
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunOFTAdapter.sol#L37

BUSINESS IMPACT

Reduced readability and maintainability, and harder external review and integration. No runtime or security impact.

RECOMMENDED NEXT STEP

Add NatSpec documentation across all contracts, prioritising the privileged setters and the cross-chain entry points, including a contract title and notice, and parameter and return descriptions on functions.

RETEST

Fixed. All six in-scope contracts now carry @title/@notice/@dev and per-function @param/@return NatSpec

I010 · CRYPTOGRAPHY (SCWE-024)

INFO

✓ FIXED

On-Chain Commit-Reveal Does Not Enforce Provable Fairness; Seed Integrity Rests on the Trusted Off-Chain Generator

DESCRIPTION

The game presents a provably-fair coin flip built from an on-chain commit-reveal and a dispute routine, but the on-chain code does not enforce that guarantee; it holds only insofar as the off-chain operator, which is outside the audited scope, generates strong and unique server seeds. The design is sound in the ordinary flow: the operator commits to a secret server seed, the player locks a client seed together with all bet picks and nonces at registration, and at settlement the operator must reveal a server seed consistent with the commitment, after which each outcome is recomputed deterministically. Because the picks are fixed at registration, the player cannot see the server seed, and the operator has committed the seed and cannot change it, an honest settlement is fair and neither side can bias it on-chain. The picks are not part of the seed commitment, so an operator cannot grind the seed against picks it does not yet know. What the contract does not provide is any enforcement of the two off-chain properties the guarantee actually depends on. First, it cannot verify the entropy of the server seed; if the off-chain seed were weak or predictable, a player could recover it from the public commitment before locking bets and choose winning picks, and the result would still settle honestly and survive dispute. Second, and more concretely, the contract performs no uniqueness check on the revealed server seed, so if the operator ever reused a seed, the player who receives the new commitment could match it against a previously revealed seed and again choose winning picks. Both paths require an off-chain operator mistake rather than an on-chain flaw, but the second is something the contract itself can and should prevent.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L147-L205
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L476-L484
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L208-L310

BUSINESS IMPACT

There is no on-chain exploit in the normal flow; the provable-fairness property the game advertises is simply not enforced on-chain and rests entirely on the trusted, out-of-scope off-chain seed generator behaving correctly. The only money-loss scenarios a player exploiting a weak or reused server seed are contingent on operator error off-chain, and the dispute routine cannot detect them because the outcomes remain internally consistent with the revealed seed. The practical, in-scope shortfall is the absence of an on-chain safeguard against server-seed reuse, together with the fact that the dispute mechanism gives an appearance of enforced fairness it does not actually deliver.

RECOMMENDED NEXT STEP

Add an on-chain check that records each revealed server seed and rejects any settlement whose server seed matches one revealed in a prior session, closing the reuse path at the contract level.

RETEST

Fixed. usedServerSeeds mapping (FunCoinflip.sol) rejects reuse at both prepareSessionCashout and settleSession, marking the seed used on first reveal.

I011 · DENIAL OF SERVICE (SCWE-140)

No Session Cancel/Expiry Strands Stake and Reservation Indefinitely

INFO

✓ FIXED

DESCRIPTION

A registered session that is never settled permanently holds the player's locked stake, now part of the bankroll, and its reservation, which shrinks the capacity for new sessions. The only path that releases the reservation and returns value requires settlement, which needs a seed preimage of the commitment. If the operator goes offline, or if the commitment is unsatisfiable, there is no player-initiated recovery and the funds and capacity are stranded. An attacker can also amplify this by registering unsatisfiable sessions to inflate the global reservation toward the bankroll, escalating it into a registration denial of service, though at a high and recoverable capital cost.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L170-L193
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L290-L292

BUSINESS IMPACT

Abandoned or unsatisfiable sessions permanently strand the locked stake and the reservation, and can be used to degrade registration capacity for all players.

RECOMMENDED NEXT STEP

Add a player-initiated cancel for an unsettled session, but gate it so it cannot be used to escape a settleable loss. Allow cancellation only after an expiry set comfortably longer than the operator's maximum settlement time, only while the session is unsettled, and give an in-flight settlement priority over a pending cancel. With those constraints the cancel reaches only genuinely abandoned sessions: a player cannot learn the outcome before the operator reveals the seed, cannot prevent the operator from settling within the window, and therefore cannot selectively cancel losing sessions. Refunding the stake and releasing the reservation of a session that never reached settlement is the correct void-the-game resolution of operator abandonment, not a reversal of a settled result. Track and release the reservation on cancellation so the capacity drain is also recovered.

RETEST

Fixed. New cancelSession (FunCoinflip.sol: player-initiated, gated on block.timestamp > registeredAt + sessionExpirySeconds, unsettled/uncancelled, and revealedServerSeed \== 0) so an in-flight cashout takes priority. It releases stake + reservation via lobby.releaseSession

I012 · BUSINESS LOGIC (SCWE-124)

Sub-Precision Dust Mishandled on the OFT Deposit and Withdrawal Paths

INFO

✓ FIXED

DESCRIPTION

Both cross-chain value paths inherit a sub-precision dust issue from the OFT's dust removal, which truncates any amount that is not a clean multiple of the decimal-conversion rate. The destination FUN token uses eight decimals against the OFT's default six shared decimals, so the conversion rate is one hundred base units and any amount carries up to ninety-nine base units of dust, on the order of a millionth of a FUN.

On the inbound deposit path, the router pulls the full intent amount from the player but the adapter sends only the dust-removed amount, so the sub-precision remainder stays in the router. The router has no sweep, so this residue is locked permanently and accrues across all deposits; it is not extractable by anyone and is not creditable to any user.

On the outbound withdrawal path, the lobby debits the player's internal balance by the full requested amount and requires the supplied send amount to equal the post-fee payout, then calls the OFT send, which removes dust. With a zero-slippage minimum the dust removal trips the slippage check and the whole withdrawal reverts atomically, forcing the player to retry with a dust-free amount. With slippage allowed the send succeeds, the player receives the dust-removed amount on the destination and forfeits the dust, which remains unattributed in the lobby and slightly improves its solvency buffer.

In both cases the magnitude is sub-precision, there is no attacker and no third-party harm, and the loss is player-unfavorable but avoidable by transacting in dust-free multiples. The degenerate deposit case, where an amount smaller than the conversion rate truncates to zero and still emits a valid zero-value compose message, is tracked separately because it carries a security consequence beyond dust.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L203-L209
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunGameLobby.sol#L451-L474
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunGameLobby.sol#L476-L498

BUSINESS IMPACT

Sub-precision FUN dust is permanently stranded in the router on every deposit that is not a clean multiple of the conversion rate, accruing over time with no recovery path, and is forfeited by the player on a dusty bridge withdrawal or else causes that withdrawal to revert under a zero-slippage minimum. All amounts are on the order of a millionth of a FUN, with no solvency impact and no attacker, and the withdrawal-side dust is retained by the lobby.

RECOMMENDED NEXT STEP

Handle dust consistently at both boundaries. On deposit, pull and approve only the dust-removed amount, refund the residual to the sender after the send, or add an owner sweep for the accumulated remainder. On withdrawal, truncate the payout to a dust-free amount before debiting the player and credit the truncated remainder back to the player's internal balance, or require dust-free amounts on the bridge-withdrawal path. A shared dust-normalisation helper applied to both paths keeps the internal accounting and the OFT's transferred amount in agreement.

RETEST

Fixed. Deposit side: the router pulls only the dust-removed bridgeAmountLD (FunBridgeSponsorRouter.sol, normalized at), so no residue accrues; sweep() covers any stray. Withdrawal side: _normalizeBridgePayout truncates the payout to a dust-free amount and credits the dust remainder back to the player's internal balance (FunGameLobby.sol); the send amount must equal the dust-free amount.

I013 · BUSINESS LOGIC (SCWE-083)

INFO

Registration Reservation Is Looser Than the Per-Bet Stake Lock (Liveness Window)

✓ FIXED

DESCRIPTION

For a fresh single bet, the registration reservation passes when the bankroll covers only the net exposure, but the first stake lock requires the bankroll to cover the full stake. In the window where the bankroll is between these two thresholds, registration clears the reservation and then reverts at the first stake lock, rolling the whole transaction back. There is no state corruption; registration simply fails until the bankroll is larger, but the looser reservation check can mislead a caller into a transaction that always reverts.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L171-L177
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunGameLobby.sol#L216-L219

BUSINESS IMPACT

Registrations in the threshold window revert atomically with no corruption, but the inconsistency between the two checks can produce confusing always-failing transactions.

RECOMMENDED NEXT STEP

Align the first-bet precondition with the registration reservation, or document the window so callers can avoid it.

RETEST

Fixed in commit b3e87b727 (PR #348). registerSession now runs _assertSequentialStakeSolvency before the reservation check it replays the per-bet stake locking (each stake must fit in houseBankroll plus the sum of prior stakes) and reverts BankrollInsufficient upfront, exactly as lockStake enforces. Registration now fails cleanly at this precondition or fully succeeds; the looser reservation check can no longer pass and then revert mid-loop at the first stake lock. Verified regression-free: the check is additive and read-only, mirrors lockStake exactly, and cannot change which registrations succeed.

I014 · GAS OPTIMIZATION

Public Constants Can Be Made Private

INFO

✓ FIXED

DESCRIPTION

The coinflip contract declares three public constants. Public constants cause the compiler to generate getter functions and method-table entries, which adds to deployment cost; their values can be read directly from the source. If external read access to these constants is not required by integrators, marking them private or internal reduces deployment gas.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunCoinflip.sol#L93-L95

BUSINESS IMPACT

Minor additional deployment gas from the auto-generated getters for public constants.

RECOMMENDED NEXT STEP

Mark constants that do not need external read access as private or internal, retaining public visibility only where integrators rely on reading them.

RETEST

Fixed
This issue has been fixed.

I015 · GAS OPTIMIZATION

Combined Revert Conditions Can Be Split for Runtime Savings

INFO

✓ FIXED

DESCRIPTION

Several guard checks combine two conditions with a logical OR inside a single revert branch. Splitting each condition into its own check trades a small increase in deployment cost for lower runtime cost when only one branch is relevant, and lets a more specific error surface. This appears in the fee-waiver guard and in two intent-validation guards.

AFFECTED CODE

- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunGameLobby.sol#L524
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L240
- https://github.com/shubhodeep1/fun-token-multi-chain/blob/30f592dd6adb44c00a677cf899578fdd84847794/contracts/FunBridgeSponsorRouter.sol#L245

BUSINESS IMPACT

Minor runtime gas, and slightly less specific revert reasons where combined conditions are used.

RECOMMENDED NEXT STEP

Split combined OR conditions into separate checks, each with its own dedicated error, where the small deployment-cost increase is acceptable.

RETEST

Fixed
This issue has been fixed.

DISCLOSURE

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or issue and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

REVIEWER(S) & CONTACT

Shreyas Koli
Security Auditor II

Prasad Kuri
Security Auditor

Neel Shah
Security Auditor

LEAD REVIEWERS & REPORT SIGNATORIES



Shashank
Co-founder & CEO · CredShields

Security analyst and penetration tester with **12 years of experience** in pentesting and **5 years** in managing bug bounty programs. Listed in **Google, Facebook, Apple,** and 30+ websites' hall of fame for reporting valid security issues.

CVE-2017-7063 CVE-2017-7062 CVE-2017-2458



Aditya Dixit
Audit & Research Lead · CredShields

Senior security researcher and penetration tester with **10+ years** in cybersecurity. Led pentest teams at **Cobalt and HackerOne**; previously at Binary.com. Executed **500+ pentests** across banking, fintech, payments, healthcare, telecom, and media, covering web, mobile (iOS/Android), network, cloud, code review, and smart contracts.

OSCP CREST CPSA AWS Security Specialty AWS Cloud Practitioner Qualys VMDR